



MNIST Classification Pipeline Phase 2 — Technical Report

Submitted by:

Omar Osama 2300090
Aly Moataz 23P0280
Abdelrahman Adel 23P0144
Loay Mahmoud 23P0419
Omar Aadly 23P0244

Submitted to:

Eng. Aly

Table of Contents

1.0	Introduction
2.0	Background & Motivation
3.0	Dataset & Preprocessing
3.1	Dataset Description
3.2	Preprocessing Pipeline
4.0	Feature Engineering
4.1	Flatten Features
4.2	PCA Features
4.3	HOG Features
5.0	Multiclass Classification Models
5.1	Decision Tree
5.2	K-Nearest Neighbors (KNN)
5.3	Multinomial Logistic Regression
5.4	Gaussian Naive Bayes
5.5	Support Vector Machine (SVM)
6.0	Hyperparameter Tuning & Cross-Validation
7.0	Regularization & Bias-Variance Analysis
8.0	Learning Curves & Overfitting/Underfitting Diagnosis
9.0	Experimental Results
10.0	Discussion & Improvement Strategies
11.0	Conclusion
12.0	Github Link

1.0 Introduction

This report documents Phase 2 of the MNIST Image Classification Pipeline project. Building on the binary classification systems developed in Phase 1, Phase 2 scales the task to full multiclass handwritten digit recognition across all ten MNIST classes (0–9). All models were implemented fully from scratch using NumPy, preserving educational transparency while achieving competitive performance.

The core objective of Phase 2 was to improve and systematically evaluate five classical machine learning models: **Decision Tree**, **K-Nearest Neighbors (KNN)**, **Multinomial Logistic Regression**, **Gaussian Naive Bayes**, and **Support Vector Machine (SVM)**. Three structured improvement strategies were applied across all models:

- **Feature Engineering** — three feature representations (Flatten, PCA, HOG) were evaluated per model to determine the optimal input space.
- **Hyperparameter Tuning** — manual grid search with K-fold cross-validation was performed to select optimal configurations.
- **Bias-Variance Diagnosis** — learning curves and train/validation gap analysis were used to detect overfitting and underfitting.

In addition, model-specific improvements were applied: ensemble methods for the Decision Tree, regularization sweeps for Logistic Regression and SVM, and variance smoothing for Naive Bayes. The report is organized into shared foundational sections covering the dataset, preprocessing, and feature engineering, followed by dedicated model sections, and concludes with a unified experimental comparison.

2.0 Background & Motivation

Multiclass classification presents fundamentally greater challenges than binary classification. While binary problems require a single decision boundary, 10-class digit recognition demands the simultaneous discrimination of ten visually overlapping categories. Digits such as 3/8, 4/9, and 5/6 share structural features that make accurate separation difficult for models relying on raw pixel inputs alone.

The MNIST dataset provides a standard benchmark for evaluating handwritten digit recognition systems. Each image captures natural variation in writing style, stroke thickness, rotation, and scale, making it a non-trivial classification problem even for classical methods. Effective classification requires meaningful feature extraction that captures structural information beyond raw pixel intensities.

Classical machine learning models — while simpler than deep networks — offer interpretability, low computational cost, and well-understood theoretical properties. Systematic evaluation of feature engineering, hyperparameter sensitivity, and generalization behavior provides insight into the strengths and limitations of each model class. This phase specifically investigates whether classical models, when paired with appropriate preprocessing pipelines, can achieve competitive accuracy on a 10-class recognition task.

3.0 Dataset & Preprocessing

3.1 Dataset Description

The MNIST (Modified National Institute of Standards and Technology) dataset is a standard benchmark for image classification. It consists of 70,000 grayscale handwritten digit images across 10 classes (digits 0–9), split into a standard training set of 60,000 images and a test set of 10,000 images.

Property	Value	Property	Value
Total Samples	70,000	Image Size	28 × 28 pixels
Training Set	60,000 images	Color Space	Grayscale (1 channel)
Test Set	10,000 images	Classes	10 (digits 0–9)
Train/Val Split	80% / 20%	Pixel Range	[0, 255] → normalized [0, 1]

3.2 Preprocessing Pipeline

A consistent preprocessing pipeline was applied across all models. The pipeline is defined as follows:

- **Normalization:** Raw pixel intensities are scaled from [0, 255] to [0.0, 1.0] by dividing by 255, centering the input distribution and improving gradient stability.
- **Train/Validation/Test Split:** The 60,000 training images are partitioned into 48,000 training samples (80%) and 12,000 validation samples (20%). The 10,000 test images are held out exclusively for final evaluation.
- **Feature Extraction:** One of three representations — Flatten, PCA, or HOG — is applied after normalization (see Section 4).
- **Standardization:** After feature extraction, features are standardized to zero mean and unit variance using statistics computed on the training set only, then applied to validation and test sets.

This pipeline ensures no data leakage between splits and provides a fair, consistent evaluation framework across all five models. Preprocessing steps are applied identically for all feature types and all models.

4.0 Feature Engineering

Feature engineering is a critical factor in classical machine learning. Three distinct feature representations were evaluated across all models. Each method encodes the input image differently, and model performance varies significantly depending on the representation chosen.

4.1 Flatten Features

The simplest representation flattens each 28×28 image into a 784-dimensional raw pixel vector:

$$x_{flat} = [x_1, x_2, \dots, x_{784}] \in \mathbb{R}^{784}$$

This preserves all original pixel information but introduces high dimensionality with correlated features, no structural encoding, and sensitivity to translation and rotation.

4.2 PCA Features

Principal Component Analysis (PCA) reduces dimensionality by projecting data onto the top-k eigenvectors of the covariance matrix. Given a centered data matrix X , PCA solves:

$$C = (1/N) X^T X, \quad C v = \lambda v$$

The projection onto the top k principal components yields:

$$x_{pca} = W_k^T x \in \mathbb{R}^k, \quad \text{where } W_k = [v_1, v_2, \dots, v_k]$$

In all experiments, $k = 50$ components were retained, preserving approximately 82.46% of variance. PCA decorrelates features and removes noise, which particularly benefits models with independence assumptions (Naive Bayes) and distance-sensitive models (KNN).

4.3 HOG Features

The Histogram of Oriented Gradients (HOG) descriptor encodes local edge and gradient orientation information. For each image cell, spatial gradients are computed and accumulated into orientation bins:

$$G_x = \partial I / \partial x, \quad G_y = \partial I / \partial y$$
$$\text{Magnitude: } |G| = \sqrt{G_x^2 + G_y^2}, \quad \text{Orientation: } \vartheta = \arctan(G_y / G_x)$$

Gradient magnitudes are accumulated into a histogram of orientations for each local cell. The resulting descriptor has 441 features. HOG captures structural shape information (strokes, edges, curves) that is more discriminative than raw pixels for handwritten digit classification.

The following table summarizes the three feature methods:

Feature Method	Dimension	Key Property		Best For
Flatten	784	Raw pixel intensities		Simple baseline; minimal preprocessing
PCA	50	Decorrelated preserving	variance-	Independence-based models; KNN
HOG	441	Gradient histograms	orientation	Structure-based; LR, SVM, Decision Tree

5.0 Multiclass Classification Models

Each model was implemented from scratch using NumPy. All five models were evaluated on the same preprocessing pipeline and the same three feature representations. Model-specific differences in improvements and optimal configurations are documented in their respective subsections.

5.1 Decision Tree

5.1.1 Mathematical Formulation

A Decision Tree classifies a sample $x \in \mathbb{R}^d$ by recursively partitioning the feature space through binary decisions. At each internal node, a split is defined by feature index j and threshold τ :

$$\text{if } x_j \leq \tau \rightarrow \text{Left Child}, \quad x_j > \tau \rightarrow \text{Right Child}$$

The predicted class at each leaf is the majority class of training samples reaching that node:

$$\hat{y} = \operatorname{argmax}_{\{k \in \{0, \dots, 9\} \mid \sum_{i \in \text{leaf}} \mathbf{1}[y_i = k]\}}$$

The split criterion uses Gini Impurity:

$$\text{Gini}(D) = 1 - \sum_k p_k^2 \quad (k = 0, \dots, 9)$$

where p_k is the class fraction at node D . The optimal split minimizes the weighted Gini across both children:

$$(j^*, \tau^*) = \operatorname{argmin}_{\{j, \tau\}} (|L|/|D| \cdot \text{Gini}(L) + |R|/|D| \cdot \text{Gini}(R))$$

Optimization is greedy recursive partitioning — no gradient descent. All features and thresholds are exhaustively searched using a vectorized one-hot prefix-sum approach. Stopping conditions: `max_depth` reached, node contains fewer than `min_samples_split` samples, or node is pure.

5.1.2 Improvements Applied

Feature Engineering Comparison

Feature Method	Accuracy	Macro Precision	Macro Recall	Macro F1
Flatten	86.07%	85.90%	85.86%	85.87%
PCA	79.41%	79.66%	79.10%	79.24%
HOG ✓	86.31%	86.27%	86.17%	86.20%

HOG features achieved the best baseline performance and were selected for all subsequent tuning experiments.

Hyperparameter Tuning — 3-Fold Cross-Validation

Manual 3-fold cross-validation was applied on 10,000 training samples to select optimal tree depth and minimum split size:

max_depth	min_samples_split	Mean CV Accuracy	Mean CV F1
8 ✓	40 ✓	79.90%	76.59%
10	30	77.29%	77.18%
12	20	77.77%	77.71%

Best configuration: max_depth = 8, min_samples_split = 40 (highest mean CV accuracy 79.90%).

Bias-Variance Analysis

Tree depth was varied across [4, 6, 8, 10, 12] to diagnose the bias-variance tradeoff:

Max Depth	Train Accuracy	Val Accuracy	Diagnosis
4	55.36%	55.76%	High bias — underfitting
6	73.69%	72.93%	Still underfitting
8 ✓	83.21%	80.90%	Best balance
10	89.99%	85.54%	Increasing variance
12	92.84%	86.31%	Mild overfitting

The final train/validation accuracy gap of 6.35% indicates mild but acceptable overfitting given the complexity of 10-class classification.

Ensemble Method — Feature-Subspace Forest

A manual feature-subspace ensemble of 5 trees was constructed, each trained on a random subset of $\sqrt{\text{total features}}$ features selected without replacement, following the standard Random Forest feature-sampling rule. Final prediction used majority voting.

5.1.3 Final Performance

Model	Accuracy	Macro Precision	Macro Recall	Macro F1
Best Baseline (HOG)	86.31%	86.27%	86.17%	86.20%
Tuned Decision Tree ✓	87.52%	87.38%	87.33%	87.34%
Feature-Subspace Ensemble	82.63%	82.77%	82.30%	82.19%

The feature-subspace ensemble scored lower than the tuned single tree (82.63% vs 87.52%). Limiting each tree to $\sqrt{d}$ features reduces individual tree quality, and majority voting cannot fully recover the lost information. Feature subspace sampling is most beneficial in high-dimensional redundant spaces, which is less the case after HOG extraction. The final tuned model achieved 87.52% test accuracy.

5.2 K-Nearest Neighbors (KNN)

5.2.1 Mathematical Formulation

KNN is a non-parametric supervised algorithm that predicts a label based on the labels of its k nearest training samples. For each test sample x , Euclidean distance to all training samples x_i is computed:

$$d(x, x_i) = \sqrt{\sum_{j=1}^n (x_j - x_{ij})^2}$$

The k nearest neighbors are selected and the predicted class is determined by majority voting:

$$\hat{y} = \underset{c}{\operatorname{argmax}} \sum_i \mathbf{1}(y_i = c) \quad \text{where } c \in \{0, \dots, 9\}$$

KNN does not optimize a loss function during training. It stores all training samples and performs inference through distance computation — making it a lazy learner.

5.2.2 Improvements Applied

Feature Engineering Comparison

Feature Method	Accuracy	Macro Precision	Macro Recall	Macro F1
Flatten	94.47%	94.49%	94.39%	94.41%
PCA ✓	96.47%	96.50%	96.43%	96.45%
HOG	93.41%	93.61%	93.32%	93.39%

PCA achieved the best performance, outperforming both Flatten and HOG. PCA reduces irrelevant noise and decorrelates features, making Euclidean distance more semantically meaningful and improving KNN's nearest-neighbor selection.

Hyperparameter Tuning — k Selection

k	Training Accuracy	Validation Accuracy	Diagnosis
1	100.00%	96.62%	Overfitting — memorizes training data
3 ✓	98.30%	96.67%	Best balance — optimal k
5	97.73%	96.59%	Slightly less flexible
7	97.40%	96.45%	Mild underfitting trend

At $k=1$, training accuracy reached 100% with a clear validation drop — classic overfitting. At large k , slight underfitting. The best bias-variance balance was achieved at $k=3$ (Validation = 96.67%).

5.2.3 Final Performance

Metric	Value
Accuracy	96.47%
Macro Precision	96.50%
Macro Recall	96.43%
Macro F1	96.45%

The confusion matrix showed strong diagonal dominance. Minor confusion occurred between visually similar digits: $7 \leftrightarrow 9$, $5 \leftrightarrow 3$, and 8 with neighboring digits. PCA produced fewer misclassifications compared to Flatten and HOG, and also achieved the fastest prediction time. Final configuration: PCA features + $k = 3$.

5.3 Multinomial Logistic Regression

5.3.1 Mathematical Formulation

Multinomial Logistic Regression (Softmax Regression) computes linear class scores for input x :

$$z = xW + b, \quad W \in \mathbb{R}^{d \times K}, \quad b \in \mathbb{R}^K$$

Softmax converts scores into class probabilities:

$$P(y = j \mid x) = e^{z_j} / \sum_{k=1}^K e^{z_k} \quad (K = 10 \text{ classes})$$

The predicted class is:

$$\hat{y} = \underset{j}{\operatorname{argmax}} P(y = j \mid x)$$

The categorical cross-entropy loss is:

$$L = - (1/N) \sum_i \sum_j y_{ij} \log(\hat{y}_{ij})$$

With L2 regularization:

$$L_{\text{reg}} = L + (\lambda / 2N) \sum W^2$$

Gradients and parameter updates:

$$\begin{aligned} \partial L / \partial W &= (1/N) X^T (\hat{Y} - Y) + (\lambda / N) W \\ W &\leftarrow W - \alpha \cdot \partial L / \partial W, \quad b \leftarrow b - \alpha \cdot \partial L / \partial b \end{aligned}$$

5.3.2 Improvements Applied

Feature Engineering Comparison

Feature Method	Accuracy	Macro F1
Flatten	90.58%	90.44%
PCA	87.09%	86.88%
HOG ✓	96.10%	96.08%

HOG achieved the best performance — improving accuracy by 5.52% over Flatten and 9.01% over PCA. Structural gradient features significantly outperform raw pixels and dimensionality-reduced representations for Logistic Regression.

Learning Rate Tuning

Learning Rate	Test Accuracy	Observation
0.001	86.63%	Underfitting — slow convergence
0.01	92.92%	Improving
0.05	95.78%	Near optimal
0.1 ✓	96.36%	Best performance — optimal

Small learning rates caused underfitting due to slow convergence. The optimal learning rate was $\alpha = 0.1$.

Regularization Analysis

L2 regularization was tested across values: $\lambda \in \{0, 0.01, 0.1, 1, 10\}$. All values produced identical test accuracy (96.36%), indicating minimal dependence on regularization due to strong feature separability provided by HOG.

5.3.3 Learning Curve Analysis

Training diagnostics over 200 epochs showed stable convergence. Loss decreased from 2.30 $\rightarrow$ 0.14 (train) and 1.85 $\rightarrow$ 0.14 (validation). Accuracy improved from 85% $\rightarrow$ 96.9% (train) and 85% $\rightarrow$ 96.3% (validation). The train-validation gap remained below 1%, indicating low variance, no overfitting, and strong generalization. High training accuracy confirms low bias.

5.3.4 Final Performance

Metric	Value	Notable
Accuracy	96.72%	Final configuration: HOG + $\alpha=0.1$
Macro Precision	96.73%	Best digit: 1 (F1 = 98.13%)
Macro Recall	96.69%	Hardest digit: 8 (F1 = 95.59%)
Macro F1	96.70%	Due to similarity with 3, 5, 9

5.4 Gaussian Naive Bayes

5.4.1 Mathematical Formulation

Gaussian Naive Bayes is a generative model governed by Bayes' Theorem. For an input x with n features, the posterior probability of class $c \in \{0, \dots, 9\}$ is:

$$P(y = c \mid x) = P(y = c) \cdot \prod_{i=1}^n P(x_i \mid y = c) / P(x)$$

Continuous features are modeled using the Gaussian probability density function:

$$P(x_i \mid y = c) = (1 / \sqrt{2\pi \sigma^2_{ci}}) \cdot \exp(- (x_i - \mu_{ci})^2 / (2\sigma^2_{ci}))$$

To prevent numerical underflow when processing high-dimensional feature vectors, classification is performed in log-space:

$$\ln P(y = c \mid x) \propto \ln P(y = c) + \sum_i \ln P(x_i \mid y = c)$$

The predicted class $\hat{y}$ is chosen by Maximum A Posteriori (MAP) decision: the class with the highest log-posterior score.

5.4.2 Improvements Applied

Feature Engineering Comparison ($\epsilon = 0.1$)

Feature Method	Accuracy	Macro F1	Weighted F1
Flatten	67.78%	66.68%	67.02%
HOG	80.54%	80.53%	80.62%
PCA (50) ✓	87.85%	87.82%	87.77%

PCA significantly outperformed the other methods. Because Naive Bayes assumes feature independence, PCA's decorrelation property allows the model to better satisfy this assumption compared to correlated raw pixels or HOG gradients.

PCA Component Sensitivity

Components (k)	Accuracy	Macro F1	Observation
30	86.66%	86.51%	Underfitting — too few components
50 ✓	87.76%	87.64%	Optimal — best performance
100	87.45%	87.35%	Slight drop — low-variance noise introduced

Accuracy peaked at $k=50$ components. Increasing to 100 introduced diminishing returns with a slight drop, suggesting inclusion of low-variance noise components.

Variance Smoothing (Regularization)

A smoothing parameter $\epsilon = 0.1$ was added to all variance estimates ($\sigma^2 \leftarrow \sigma^2 + \epsilon$) to ensure numerical stability and prevent division-by-zero errors during Gaussian probability computation. This acts as a regularization term preventing degenerate distributions from dominating log-posterior scores.

5.4.3 Learning Curve Analysis

Training and validation accuracy converged to approximately 86.9%, with the absence of a generalization gap confirming low variance and strong robustness against overfitting. The early plateau after $\sim 10,000$ samples demonstrates that Naive Bayes is an efficient learner that reaches its optimal capacity quickly.

5.4.4 Final Performance

Metric	Value	Notable
Accuracy	87.85%	Final config: PCA (50), $\epsilon = 0.1$
Macro Precision	87.98%	Best digit: 1 ($F1 \approx 0.95$) — unique vertical shape
Macro Recall	87.78%	Hardest digits: 5, 8 ($F1 \approx 0.78\text{--}0.83$)
Macro F1	87.82%	Visual similarity in PCA feature space

5.5 Support Vector Machine (SVM)

5.5.1 Mathematical Formulation

The multiclass SVM uses the One-vs-Rest (OvR) strategy. For each binary subproblem, binary labels are assigned:

$$y_i \in \{-1, +1\}$$

The linear SVM decision function and prediction rule:

$$f(x_i) = w^T x_i + b, \quad \hat{y}_i = \text{sign}(f(x_i))$$

Hinge loss for one sample:

$$L_i = \max(0, 1 - y_i(w^T x_i + b))$$

Soft-margin SVM objective:

$$J(w, b) = (1/2) \|w\|^2 + C \cdot (1/n) \sum_i \max(0, 1 - y_i(w^T x_i + b))$$

For mini-batch gradient descent, let $V = \{i : y_i(w^T x_i + b) < 1\}$ be the margin-violating set. Gradients:

$$\nabla_w J = w - C \cdot (1/m) \sum_{i \in V} y_i x_i$$

$$\nabla_b J = -C \cdot (1/m) \sum_{i \in V} y_i$$

Parameter updates: $w \leftarrow w - \eta \nabla_w J$, $b \leftarrow b - \eta \nabla_b J$

For multiclass, one binary classifier is trained per class k :

$$y_i^{(k)} = +1 \text{ if } y_i = k, \quad -1 \text{ otherwise}$$

Each classifier produces score $s_k(x) = w_k^T x + b_k$. Final prediction:

$$\hat{y} = \operatorname{argmax}_{\{k \in \{0, \dots, 9\}\}} s_k(x)$$

5.5.2 Justification for Feature Selection

Flattened 784-dimensional pixels were not used for SVM. Raw pixels do not explicitly capture edges, stroke directions, or local shape patterns, making SVM optimization over a high-dimensional correlated space computationally expensive with degraded margin quality. PCA and HOG were selected based on Phase 1 investigation; HOG significantly outperformed PCA in Phase 2 experiments and was selected for the final model.

5.5.3 Improvements Applied

Baseline Feature Comparison

Baseline used: $C=1.0$, learning rate=0.001, 20 epochs, batch size=512.

Feature	Accuracy	Macro F1	Weighted F1
PCA	80.93%	80.27%	80.40%
HOG ✓	91.76%	91.72%	91.73%

Hyperparameter Tuning

C	Learning Rate	Epochs	Weighted F1
0.1	0.001	20	86.22%
1.0	0.001	20	91.73%
5.0	0.001	20	95.81%
10.0 ✓	0.001	20	96.47%
1.0	0.003	20	93.77%
1.0	0.0005	40	91.72%

Regularization Analysis

C	Training Acc.	Validation Acc.	Observation
0.01	85.01%	85.12%	Underfitting — over-regularized
0.1	86.03%	86.26%	Still underfitting
1.0	92.00%	91.76%	Balanced
5.0	96.04%	95.82%	Good balance
10.0	96.70%	96.47%	Best from tuning
20.0 ✓	97.13%	96.83%	Final selection — no gap increase

Increasing C continued to improve validation accuracy up to C=20 without widening the train-validation gap; C=20 was selected for the final model.

5.5.4 Learning Curves

Training Size	Training Acc.	Validation Acc.	Gap
5,000	96.04%	94.42%	1.62%
10,000	96.50%	95.68%	0.82%
20,000	96.85%	96.36%	0.49%
40,000	97.14%	96.83%	0.31%
48,000	97.14%	96.88%	0.26%

The training-validation gap narrowed as training size increased, confirming good generalization and no strong overfitting. The SVM benefits from more data.

5.5.5 Final Performance

Accuracy	Macro Precision	Macro Recall	Macro F1	Weighted F1
96.91%	96.90%	96.88%	96.89%	96.91%

Per-Class Performance:

Class	Precision	Recall	F1
0	96.12%	98.67%	97.38%
1	97.74%	99.21%	98.47%
2	97.09%	97.09%	97.09%
3	97.14%	97.43%	97.28%
4	98.24%	96.54%	97.38%
5	96.95%	96.30%	96.63%
6	97.40%	97.60%	97.50%
7	96.18%	95.43%	95.80%
8	96.36%	95.17%	95.76%
9	95.82%	95.34%	95.58%

6.0 Hyperparameter Tuning & Cross-Validation

All models were tuned using a manual K-fold cross-validation procedure. For each hyperparameter configuration θ , training data was partitioned into $K=3$ disjoint folds F_1, F_2, F_3 . For each fold F_i , the model was trained on $\bigcup_{j \neq i} F_j$ and evaluated on F_i . The mean cross-validation score was computed as:

$$CV(\vartheta) = (1/K) \sum_{i=1}^K a_i$$

The optimal configuration was selected as:

$$\vartheta^* = \operatorname{argmax}_{\vartheta} CV(\vartheta)$$

This strategy provides a more reliable performance estimate than a single validation split, particularly for models with high variance or complex hyperparameter interactions. The following table summarizes the best hyperparameters identified per model:

Model	Parameter(s) Tuned	Values Tested	Best Value	Metric Used
Decision Tree	max_depth, min_samples_split	[8,10,12] [20,30,40]	× depth=8, split=40	CV Accuracy

Model	Parameter(s) Tuned	Values Tested	Best Value	Metric Used
KNN	k (neighbors)	1, 3, 5, 7	k = 3	Val Accuracy
Logistic Regression	Learning rate α	0.001, 0.01, 0.05, 0.1	$\alpha = 0.1$	Test Accuracy
Naive Bayes	PCA components, ϵ	$k \in \{30, 50, 100\}$, $\epsilon \in \{0.01, 0.1, 1\}$	k=50, $\epsilon=0.1$	Accuracy
SVM	C, learning rate	$C \in [0.01, 20]$, $\eta = 0.001$	C=20, $\eta=0.001$	Weighted F1

7.0 Regularization & Bias-Variance Analysis

Regularization was explicitly analyzed in the Logistic Regression and SVM models. In both cases, the regularization term penalizes large weight magnitudes, controlling model complexity and the bias-variance tradeoff.

L2 Regularization (Logistic Regression)

The regularized loss is:

$$L_{reg} = L + (\lambda/2N) \sum W^2$$

Testing $\lambda \in \{0, 0.01, 0.1, 1, 10\}$ on HOG features all produced identical test accuracy of 96.36%, indicating that HOG features are highly separable and the model is largely insensitive to regularization strength within this range.

L2 Regularization (SVM)

The SVM objective includes the regularization term:

$$L_{reg} = (\lambda/2) \|W\|_F^2, \text{ where } \|W\|_F^2 = \sum_{k,j} W_{kj}^2$$

The parameter $C = 1/\lambda$ controls the margin-error trade-off. Key findings:

- Small C (strong regularization): simpler model, possible underfitting — training and test accuracy both low.
- Large C (weak regularization): more focus on reducing training errors — risk of overfitting but train-test gap remains small here due to strong HOG features.
- C=20 was optimal: test accuracy 96.91%, train-test gap <0.3%.

For the Decision Tree, depth acts as an implicit regularization parameter. Shallow trees (depth ≤ 6) underfitted; depth 8 provided the best bias-variance balance with a 6.35% train-validation gap.

For Naive Bayes, variance smoothing $\epsilon = 0.1$ provided numerical regularization. For KNN, k effectively controls smoothing — small k overfits (memorizes data), large k underfits by averaging too broadly.

8.0 Learning Curves & Overfitting/Underfitting Diagnosis

Learning curve analysis was performed for all models by recording training and validation accuracy as a function of training set size. Diagnostic heuristics:

- Gap > 5%: Possible overfitting (train accuracy significantly exceeds validation).
- Train accuracy < 80% AND Val accuracy < 80%: Possible underfitting.
- Otherwise: Good generalization.

Model	Low Behavior	Data	Full Data Acc.	Train	Full Val/Test Acc.	Data	Gap	Diagnosis
Decision Tree	High bias at depth ≤ 6		92.84%		86.31%		6.35%	Mild overfitting — acceptable
KNN	$k=1$ memorizes training		98.30%		96.67%		1.63%	Good generalization ($k=3$)
Logistic Regression	Slow convergence at low α		96.90%		96.30%		<1%	Low variance — excellent
Naive Bayes	Early plateau ~10k samples		~87%		~86.9%		<0.5%	Low variance — efficient learner
SVM	Improving with more data		97.14%		96.88%		0.26%	Strong generalization

Key observations: Logistic Regression and SVM showed the smallest train-validation gaps, confirming strong generalization. The Decision Tree showed the largest gap (6.35%) due to the inherent tendency of unpruned trees to overfit. Naive Bayes converged fastest, reaching near-optimal performance with only ~10,000 training samples — reflecting its parametric efficiency as a generative model.

9.0 Experimental Results

All models were evaluated on the held-out test set using their optimal configurations identified through cross-validation and tuning. The following table provides a unified comparison of final performance:

Model	Best Feature	Accuracy	Macro Precision	Macro Recall	Macro F1
Decision Tree	HOG	87.52%	87.38%	87.33%	87.34%
KNN	PCA	96.47%	96.50%	96.43%	96.45%
Logistic Regression	HOG	96.72%	96.73%	96.69%	96.70%
Naive Bayes	PCA	87.85%	87.98%	87.78%	87.82%
SVM	HOG	96.91%	96.90%	96.88%	96.89%

SVM achieved the highest overall accuracy (96.91%), followed closely by Logistic Regression (96.72%) and KNN (96.47%). These three models significantly outperformed Decision Tree (87.52%) and Naive Bayes (87.85%).

Feature Method Summary:

- HOG was optimal for Decision Tree, Logistic Regression, and SVM — capturing structural gradient information that benefits discriminative boundary-based models.
- PCA was optimal for KNN and Naive Bayes — decorrelated features improve Euclidean distance semantics for KNN and satisfy the independence assumption for Naive Bayes.
- Flatten generally underperformed HOG and PCA across all models.

10.0 Discussion & Improvement Strategies

Effect of Feature Engineering

Feature engineering was the single most impactful factor across all models. HOG provided the best performance for discriminative models (SVM, LR, DT) by encoding structural gradient information that captures stroke direction and edge patterns — directly relevant to digit shape. PCA benefited generative and distance-based models (Naive Bayes, KNN) by decorrelating features and reducing noise dimensionality. Raw pixel flattening was consistently the weakest representation, despite being highest-dimensional, confirming that feature quality dominates over quantity.

Strengths and Limitations by Model

SVM:

Strongest overall performer. The OvR hinge-loss formulation provides theoretically well-grounded margin maximization. Mini-batch SGD allows scalability. Main limitation: from-scratch implementation with sample-level updates is slower than optimized solvers.

Logistic Regression:

Smooth, differentiable cross-entropy objective converges reliably with gradient descent. Near-SVM performance with simpler optimization. Full-batch updates are memory-efficient at this scale but do not generalize to very large datasets.

KNN:

Benefits enormously from PCA's dimensionality reduction — Euclidean distance in pixel space is not semantically meaningful, while PCA space improves class separability. Main limitation: $O(N \cdot d)$ inference cost does not scale to large datasets.

Naive Bayes:

Most computationally efficient — reaches near-optimal capacity quickly and requires no iterative optimization. PCA's decorrelation partially satisfies the feature independence assumption. Performance ceiling is lower due to the conditional independence assumption being inherently violated by structured image data.

Decision Tree:

Interpretable and fast to train. Feature subspace ensembling did not improve over the tuned single tree, suggesting HOG features are sufficiently non-redundant that random feature sampling hurts individual tree quality. Most sensitive to overfitting among the five models.

Computational Tradeoffs

Decision Tree and Naive Bayes were fastest to train. KNN has zero training time but expensive inference. Logistic Regression and SVM required iterative optimization, with SVM being slowest due to 10 binary classifiers trained per configuration. PCA's reduced dimensionality (50 vs 784) provided significant speed benefits for KNN and Naive Bayes.

11.0 Conclusion

This report presented a systematic extension of Phase 1 binary classifiers to full 10-class multiclass digit recognition on MNIST. Five classical machine learning models — Decision Tree, KNN, Multinomial Logistic Regression, Gaussian Naive Bayes, and One-vs-Rest Linear SVM — were implemented from scratch using NumPy and evaluated through a rigorous pipeline of feature engineering, hyperparameter tuning, regularization analysis, and learning curve diagnosis.

The key findings are:

- Feature engineering was the dominant performance factor. HOG features consistently outperformed raw pixels and PCA for discriminative models, while PCA was superior for KNN and Naive Bayes.
- SVM achieved the best overall performance at 96.91% test accuracy and 96.89% Macro F1, followed by Logistic Regression (96.72%) and KNN (96.47%).
- Decision Tree and Naive Bayes achieved 87.52% and 87.85% respectively — competitive given their model simplicity, but limited by structural assumptions (greedy splitting, feature independence).
- Hyperparameter tuning via cross-validation yielded measurable improvements in all models. The Decision Tree improved from 86.31% to 87.52% test accuracy through depth optimization.
- All models showed good generalization. The SVM and Logistic Regression exhibited the smallest train-validation gaps (<1%), confirming effective bias-variance management.

The results demonstrate that classical machine learning methods remain highly effective for multiclass image classification when combined with thoughtful feature engineering and systematic optimization. The performance gap between models is largely explained by how well each model's assumptions align with the chosen feature representation — a finding that underscores the importance of the feature engineering–model compatibility analysis conducted in this phase.

12.0 Github Link

[AbdelrahmanAdel-1/MNIST-ML-ImageClassifiers: Binary & Multi-class Image Classification on MNIST using classical ML algorithms — built from scratch](#)